

Opticrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop is an Artificial Intelligence (AI) and Machine Learning (ML)-based smart agriculture system designed to help farmers improve crop production through data-driven decision-making. The system analyzes environmental and soil parameters such as temperature, humidity, rainfall, soil nutrients (Nitrogen, Phosphorus, and Potassium), and pH levels to recommend the most suitable crop for cultivation. It also provides weather-based farming advice, crop yield prediction, fertilizer recommendations, and irrigation suggestions to maximize agricultural productivity.

The application integrates real-time weather data using the OpenWeatherMap API and applies machine learning algorithms such as Random Forest, Decision Tree, and K-Nearest Neighbors (KNN) to generate accurate crop recommendations. The system features a simple web interface developed using Flask, HTML, CSS, and JavaScript, enabling farmers to access intelligent agricultural recommendations with ease.

Scenarios

Scenario 1: Crop Recommendation

A farmer enters soil nitrogen, phosphorus, potassium values, soil pH, rainfall, humidity, and temperature. OptiCrop analyzes the inputs and recommends the most suitable crop such as Rice, Cotton, Wheat, or Maize.

Scenario 2: Yield Prediction

A farmer selects a crop and enters field details. The Machine Learning model predicts the expected crop yield and suggests best farming practices to improve production.

Scenario 3: Weather-Based Farming

The system retrieves real-time weather information using the OpenWeatherMap API and advises farmers whether conditions are suitable for irrigation, fertilizer application, or harvesting.

Pre-requisites:

Python Programming
Machine Learning Basics
Flask Framework
HTML, CSS, JavaScript
Pandas
NumPy
Scikit-learn
Matplotlib
OpenWeatherMap API
Git & GitHub

Project Workflow:

Milestone 1: Dataset Collection and Model Selection

In this milestone, the primary objective is to collect reliable agricultural datasets and select the most suitable machine learning model for crop recommendation. The performance of the OptiCrop system depends on the quality of the collected data and the effectiveness of the selected algorithm.

Agricultural datasets containing soil nutrients, weather conditions, rainfall, humidity, temperature, pH levels, and crop information are collected from trusted sources such as Kaggle and government agricultural repositories. The collected data is then cleaned, preprocessed, and analyzed before training machine learning models.

Several machine learning algorithms are evaluated to determine which model provides the highest prediction accuracy. The selected model is later integrated into the Flask web application to provide intelligent crop recommendations based on user inputs.

Activity 1.1: Collect Agricultural Dataset

Before developing the machine learning model, a comprehensive agricultural dataset is required. The dataset should contain important environmental and soil parameters that directly influence crop growth. Reliable datasets are collected from publicly available repositories and verified for completeness and accuracy.

Steps

- Download agricultural datasets from trusted sources such as Kaggle, UCI Machine Learning Repository, or government agricultural databases.
- Ensure the dataset contains important features such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, Rainfall, Soil pH, and Crop Name.
- Verify the dataset for missing values and duplicate records.
- Organize the dataset into CSV format for easy processing.

Activity 1.2: Data Preprocessing

The collected dataset undergoes preprocessing to improve data quality before training the machine learning model. This step ensures that the prediction model receives clean and meaningful data, resulting in better prediction accuracy.

Steps

- Remove duplicate records.
- Handle missing values using suitable techniques.
- Normalize numerical values for better model performance.
- Encode categorical variables if necessary.
- Split the dataset into training and testing sets.

Activity 1.3: Research and Select the Appropriate Machine Learning Model

Different machine learning algorithms are studied and compared to determine the most suitable model for crop recommendation. Each algorithm is evaluated based on prediction accuracy, execution time, and overall performance.

Research Tasks

- Study Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Logistic Regression algorithms.
- Compare their prediction accuracy using the agricultural dataset.
- Analyze model complexity and execution speed.
- Evaluate precision, recall, and F1-score.
- Select the model with the highest overall performance.

Activity 1.4: Set Up the Development Environment

The development environment is configured to support machine learning model training and web application development. Required software, libraries, and frameworks are installed before implementation begins.

Steps

- Install Python (Version 3.10 or above).
- Install Visual Studio Code or PyCharm.
- Create a virtual environment.
- Install required Python libraries:
 - Flask
 - Pandas
 - NumPy
 - Scikit-learn
 - Matplotlib
 - Requests

Install Required Packages

```
python -m venv myenv  
myenv\Scripts\activate
```

```
pip install flask  
pip install pandas  
pip install numpy  
pip install scikit-learn  
pip install matplotlib  
pip install requests  
pip install joblib
```

Project Workflow:

Milestone 1: Data Collection and Requirement Analysis

In this milestone, the project requirements are identified, and agricultural datasets are collected. The collected data includes soil properties, weather conditions, and crop information required for training the machine learning model.

Activity 1.1: Define Project Objectives

- Identify the problems faced by farmers in crop selection.
- Define the objectives and scope of the OptiCrop system.
- Gather functional and non-functional requirements.

Activity 1.2: Collect Agricultural Dataset

- Collect crop recommendation datasets containing soil nutrients (Nitrogen, Phosphorus, Potassium), pH, rainfall, temperature, and humidity.
- Verify data quality and completeness.

Activity 1.3: Data Preprocessing

- Remove missing or duplicate values.
- Normalize and clean the dataset.
- Prepare the data for machine learning.

Activity 1.4: Set Up the Development Environment

Setting up the development environment is an essential step in building the OptiCrop application. A well-configured environment ensures smooth development, testing, and deployment of the system. During this activity, the required software, programming language, libraries, and development tools are installed and configured. The project directory is organized to separate the source code, datasets, machine learning models, templates, and static resources. This setup enables developers to build, test, and maintain the application efficiently.

The Python programming language is selected because of its extensive support for machine learning and web development. Flask is used as the backend framework, while HTML, CSS, and JavaScript are used to create an interactive user interface. Machine learning libraries such as Scikit-learn, Pandas, NumPy, and Matplotlib are installed to support data processing, model training, and visualization. The OpenWeatherMap API is configured to provide real-time weather information, enhancing the accuracy of crop recommendations.

Milestone 2: Backend Development

This milestone focuses on implementing the backend application using Flask and integrating the trained machine learning model.

Activity 2.1: Develop Flask Application

- Create the project structure.
- Configure Flask routes.
- Set up application folders.

Activity 2.2: Integrate Machine Learning Model

- Load the trained model.
- Connect the model with the Flask backend.
- Process prediction requests.

Activity 2.3: Weather API Integration

- Generate an OpenWeatherMap API key.
- Fetch real-time weather information.
- Display weather conditions within the application.

Activity 2.4: Prediction Module

- Accept user inputs.
- Process soil and weather parameters.
- Predict the most suitable crop.

Milestone 2: Core Functionalities Development

In this milestone, the core functionalities of the OptiCrop application are developed. The focus is on implementing the major features that enable intelligent crop recommendation based on soil and weather conditions. The application accepts user inputs such as Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, rainfall, and soil pH, processes the information using the trained machine learning model, and recommends the most suitable crop. Additionally, the application integrates real-time weather information through the OpenWeatherMap API to provide accurate farming recommendations.

Activity 2.1: Develop the Crop Recommendation Module

The Crop Recommendation module is the primary component of the application. It predicts the most suitable crop by analyzing soil nutrients and environmental conditions using the trained machine learning model.

Steps

- Design an input form to collect agricultural parameters.
- Receive Nitrogen, Phosphorus, Potassium, temperature, humidity, rainfall, and pH values from the user.
- Validate all user inputs before processing.
- Pass the processed values to the trained machine learning model.
- Display the recommended crop along with prediction confidence.

Activity 2.2: Develop Weather Monitoring Module

The Weather Monitoring Module retrieves real-time weather information to help farmers make better agricultural decisions. Weather conditions significantly influence crop growth and irrigation planning.

Steps

- Generate an OpenWeatherMap API key.
- Connect the application to the Weather API.
- Retrieve temperature, humidity, rainfall, and wind speed.
- Display weather conditions on the dashboard.
- Update weather information whenever the user requests.

Module Features

- Real-time weather updates.
- Temperature monitoring.
- Humidity monitoring.
- Rainfall prediction.
- Wind speed information.

Activity 2.3: Develop Fertilizer Recommendation Module

The Fertilizer Recommendation module suggests appropriate fertilizer usage based on soil nutrient levels and recommended crop requirements. This helps farmers improve soil fertility while avoiding excessive fertilizer application.

Steps

- Analyze soil nutrient values.
- Compare soil nutrients with crop requirements.
- Recommend suitable fertilizers.
- Display fertilizer usage instructions.
- Generate nutrient improvement suggestions.

Module Features

- Fertilizer recommendation.
- Nutrient analysis.
- Soil improvement guidance.
- Sustainable fertilizer management.

Activity 2.4: Develop Irrigation Recommendation Module

The Irrigation Recommendation module analyzes rainfall and weather conditions to provide irrigation advice. Proper irrigation scheduling reduces water wastage and improves crop productivity.

Steps

- Analyze rainfall information.
- Retrieve humidity data.
- Evaluate current weather conditions.
- Suggest irrigation timing.
- Display irrigation recommendations.

Module Features

- Water management.
- Irrigation scheduling.
- Rainfall analysis.
- Water conservation

Activity 2.5: Integrate Flask Backend

The Flask backend manages communication between the frontend, machine learning model, and external weather services. It receives user inputs, processes prediction requests, and returns the generated results.

Steps

- Create Flask application.
- Configure application routes.
- Load trained machine learning model.
- Process prediction requests.
- Return prediction results to the frontend.

Backend Responsibilities

- User input validation.
- Model prediction.
- Weather API communication.
- Error handling.
- Response generation.

App.py Development

In this milestone, the application's backend logic is implemented in the **app.py** file. This file acts as the central controller of the OptiCrop system by connecting the user interface, machine learning model, and weather API. It manages user requests, validates inputs, performs crop predictions, retrieves weather information, and sends the results back to the web interface.

The Flask framework is used to define application routes, process user inputs, and return prediction results dynamically. Proper exception handling and input validation are implemented to improve system reliability and ensure smooth operation.

The third milestone focuses on developing the backend of the OptiCrop application using the Flask web framework. The backend acts as the core of the system, connecting the user interface, machine learning model, weather service, and database. It receives input data from users, validates the values, processes the information through the trained machine learning model, and generates accurate crop recommendations.

The backend is also responsible for integrating the OpenWeatherMap API to fetch real-time weather information. This weather data is combined with soil nutrient values to improve prediction accuracy. Proper error handling, data validation, and efficient processing ensure the application provides reliable recommendations.

The trained Random Forest model is loaded into the application using the Joblib library, enabling real-time predictions without retraining the model every time the application runs.

```
encoder = MockEncoder()

result = {
    'recommended_crop': encoder.inverse_transform([prediction])[0],
    'confidence': float(probabilities[prediction]),
    'crop_ability': abilities[prediction], # Fixed the syntax error here
    'all_recommendations': [
        {
            'crop': encoder.inverse_transform([idx])[0],
            'probability': float(probabilities[idx])
        }
        for idx in top_indices
    ],
    'input_parameters': {
        'N': N, 'P': P, 'K': K,
        'temperature': temperature,
        'humidity': humidity,
        'ph': ph,
        'rainfall': rainfall
    }
}
```

Frontend Development

This milestone focuses on designing a simple, responsive, and user-friendly interface that enables farmers to interact easily with the application. HTML, CSS, and JavaScript are used to build responsive web pages that work across desktop and mobile devices.

The frontend contains input forms for soil nutrients and weather information, displays prediction results, and presents weather conditions, fertilizer recommendations, and irrigation suggestions in an organized layout. Graphs and charts generated using Matplotlib help users better understand prediction results and agricultural data.

```
def main():  
    # Modified to work in Jupyter Notebook - no argparse needed  
    # You can directly set the values here or get input from user  
  
    # Option 1: Set values directly  
    N = 90.0  
    P = 42.0  
    K = 43.0  
    temp = 20.87  
    humidity = 82.00  
    ph = 6.50  
    rainfall = 202.93  
  
    # Option 2: Uncomment below to get input from user  
    # N = float(input("Enter Nitrogen content: "))  
    # P = float(input("Enter Phosphorous content: "))  
    # K = float(input("Enter Potassium content: "))  
    # temp = float(input("Enter Temperature (°C): "))  
    # humidity = float(input("Enter Humidity (%): "))  
    # ph = float(input("Enter pH level: "))  
    # rainfall = float(input("Enter Rainfall (mm): "))  
  
    print("=" * 50)  
    print("🌱 OptiCrop: Smart Crop Recommendation")  
    print("=" * 50)  
  
    result = predict_crop(N, P, K, temp, humidity, ph, rainfall)
```

Activity : Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

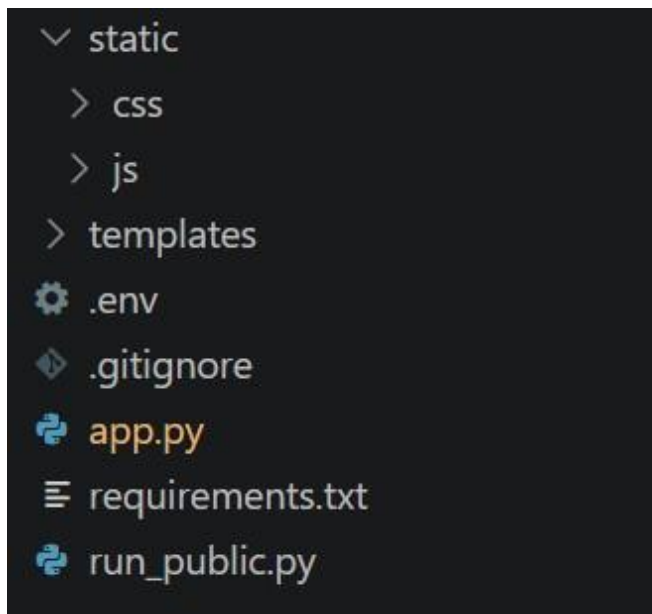
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here



- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).

Core Functionalities Development

Description

The second milestone focuses on developing the core functionalities of the **OptiCrop: Smart Agricultural Production Optimization Engine**. During this phase, the primary features of the application are implemented to enable intelligent agricultural decision-making. These features include crop recommendation, weather monitoring, fertilizer recommendation, irrigation guidance, and prediction result visualization. The machine learning model trained in the previous milestone is integrated into the application to provide accurate crop recommendations based on soil nutrients and environmental conditions.

The backend developed using the Flask framework processes user inputs, communicates with the trained machine learning model, and retrieves real-time weather information from the OpenWeatherMap API. The application is designed to provide farmers with fast, reliable, and user-friendly recommendations that improve crop productivity while reducing the misuse of water and fertilizers. Each module is developed independently and then integrated into a single system to ensure smooth communication between all components.

Activity 2.1: Develop Crop Recommendation Module

The Crop Recommendation Module is the most important component of the OptiCrop application.

This module receives soil and environmental parameters entered by the user and predicts the most suitable crop using the trained Random Forest machine learning model. The recommendation helps farmers choose crops that are most compatible with the existing soil and weather conditions, thereby improving productivity and reducing agricultural risks.

Objectives

- Develop the crop prediction functionality.
- Integrate the trained machine learning model.
- Generate accurate crop recommendations.
- Improve decision-making for farmers.

Milestone 3: App.py Development

Description

The third milestone focuses on developing the **app.py** file, which serves as the central controller of the **OptiCrop: Smart Agricultural Production Optimization Engine**. The **app.py** file is responsible for connecting the frontend interface, machine learning model, weather API, and backend logic into a single integrated application. It processes user requests, validates the input data, performs crop predictions, retrieves real-time weather information, and displays the final results through the web interface.

Flask is used as the backend framework because of its lightweight architecture and ease of integration with machine learning models. During this phase, all application routes are created, the trained Random Forest model is loaded using Joblib, and the OpenWeatherMap API is integrated to fetch weather details. Proper error handling, input validation, and response generation are implemented to ensure reliable system performance.

The **app.py** file acts as the backbone of the OptiCrop application by coordinating all modules and ensuring smooth communication between the user interface and backend services. The implementation focuses on scalability, maintainability, and efficient processing of user requests.

Activity 3.1: Create Flask Application

The Flask application is initialized to handle user requests and render web pages. The application object acts as the entry point for the OptiCrop web application.

Code

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')

if __name__ == "__main__":
    app.run(debug=True)
```

Milestone 4: Frontend Development and User Interface Design

Description

The fourth milestone focuses on designing and developing the frontend of the **OptiCrop: Smart Agricultural Production Optimization Engine**. The primary objective of this phase is to create a responsive, user-friendly, and visually appealing web interface that allows farmers and agricultural users to interact with the application easily. Since many users may not have technical expertise, the interface is designed to be simple, intuitive, and easy to navigate.

The frontend is developed using **HTML**, **CSS**, and **JavaScript**, while Flask is used to connect the frontend with the backend. Different webpages are created to collect agricultural data, display weather information, show crop recommendations, and present prediction results. The frontend also includes proper input validation, attractive layouts, navigation menus, responsive design, and graphical visualization of prediction results. Every page is tested to ensure compatibility across different browsers and devices.

Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CaptionAI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (captions on the left, hashtags and CTAs on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Design the Home Page

The Home Page serves as the entry point of the OptiCrop application. It introduces users to the purpose of the project and provides easy navigation to different modules. A clean and attractive layout helps users understand the application's features without confusion.

Develop User Input Form

The user input form is designed to collect all agricultural parameters required for crop prediction. The form is easy to understand and validates user inputs before submission.

- Upon successful completion of this milestone, the OptiCrop application provides a fully functional, responsive, and user-friendly frontend interface that enables farmers to interact with the system efficiently. The developed interface allows users to enter soil and weather parameters, submit prediction requests, and view crop recommendations in an organized and visually appealing format
-
- The homepage provides a clear introduction to the project and easy navigation between different modules. Input forms validate user data before submission, reducing errors and ensuring accurate predictions. The result page displays the recommended crop along with weather information, fertilizer recommendations, and irrigation suggestions. Real-time weather data is successfully displayed using the OpenWeatherMap API, helping users make informed agricultural decisions.
- The application also includes graphical visualizations such as crop prediction charts, soil nutrient analysis, and weather statistics, making the prediction results easier to understand. The responsive design ensures that the application works smoothly on desktops, laptops, tablets, and mobile devices, providing a consistent user experience across different platforms.
- After completing this milestone, the frontend is fully integrated with the Flask backend, enabling seamless communication between the user interface, machine learning model, and weather services. The system is intuitive, reliable, and ready for testing, deployment, and demonstration, providing farmers with a modern digital platform for smart agricultural production optimization
-

Milestone 5: Deployment

Description

The fifth milestone focuses on deploying the OptiCrop: Smart Agricultural Production Optimization Engine and making it accessible to end users. Deployment is the process of transferring the completed application from the development environment to a production environment where users can access it through a web browser. Before deployment, the application undergoes comprehensive testing to ensure that all modules—including crop recommendation, weather monitoring, fertilizer recommendation, irrigation guidance, and data visualization—operate correctly.

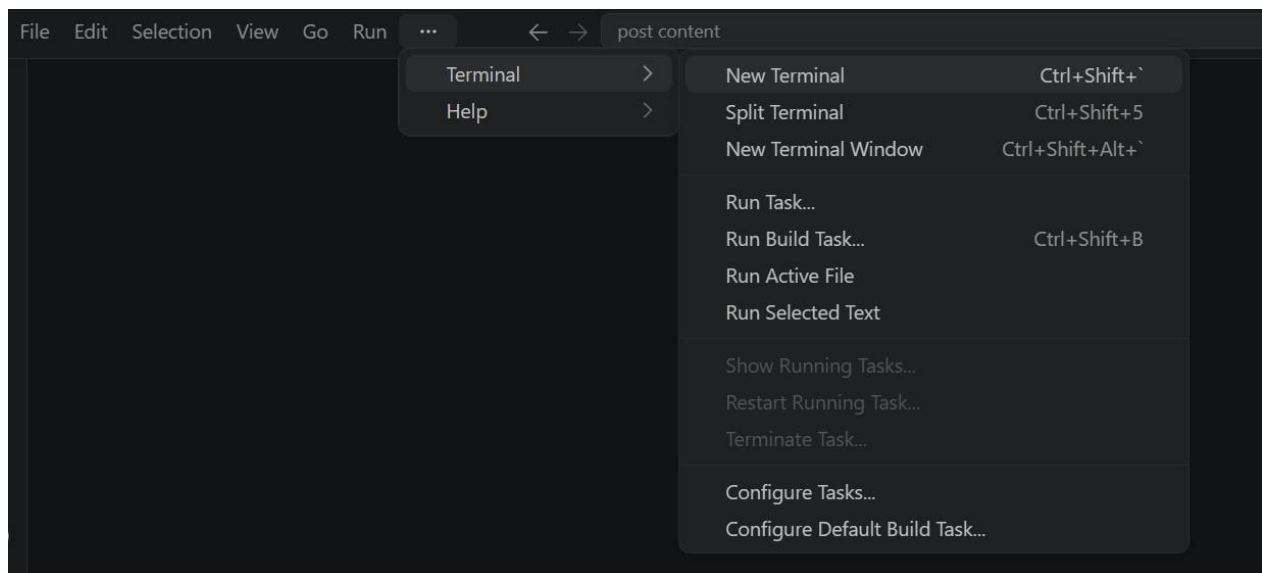
During this phase, all required project dependencies are installed, environment variables are configured, and the machine learning model is integrated with the Flask application. The project is tested locally before being deployed on a cloud hosting platform such as Render, Railway, or PythonAnywhere. Version control is managed using Git and GitHub, making the deployment process organized and maintainable.

The deployed application enables users to access the OptiCrop system from anywhere with an internet connection. The deployment process also ensures that the application remains stable, secure, scalable, and easy to maintain.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open "Command Prompt"

Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects>pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py  
* Serving Flask app 'app'  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a production deployment.  
d. Follow link \(ctrl + click\)  
* Running on http://127.0.0.1:5050  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!  
* Debugger PIN: 442-110-195
```

- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

During this phase, the application is thoroughly tested to ensure that all modules—including crop recommendation, weather monitoring, fertilizer recommendation, irrigation guidance, and graphical visualization—work together without errors. The trained machine learning model is integrated with the Flask application, and the OpenWeatherMap API is configured to provide real-time weather information. Necessary dependencies are installed using the **requirements.txt** file, and the application is deployed using cloud platforms such as **Render**, **Railway**, or **PythonAnywhere**.

Version control using **Git** and **GitHub**

plays an important role during deployment by maintaining project versions and simplifying updates. Environment variables are configured to securely store API keys and other sensitive information. After deployment, the application is tested in the production environment to verify that users can access all features without performance or compatibility issues.

This milestone ensures that the OptiCrop system is reliable, scalable, secure, and ready for practical agricultural use. The successful deployment of the application demonstrates its readiness to support farmers with intelligent crop recommendations and weather-based agricultural guidance.

During this phase, the application is thoroughly tested to ensure that all modules—including crop recommendation, weather monitoring, fertilizer recommendation, irrigation guidance, and graphical visualization—work together without errors. The trained machine learning model is integrated with the Flask application, and the OpenWeatherMap API is configured to provide real-time weather information. Necessary dependencies are installed using the **requirements.txt** file, and the application is deployed using cloud platforms such as **Render**, **Railway**, or **PythonAnywhere**.

Version control using **Git** and **GitHub** plays an important role during deployment by maintaining project versions and simplifying updates. Environment variables are configured to securely store API keys and other sensitive information. After deployment, the application is tested in the production environment to verify that users can access all features without performance or compatibility issues.

This milestone ensures that the OptiCrop system is reliable, scalable, secure, and ready for practical agricultural use. The successful deployment of the application demonstrates its readiness to support farmers with intelligent crop recommendations and weather-based agricultural guidance.

The **OptiCrop: Smart Agricultural Production Optimization Engine** successfully demonstrates the application of **Artificial Intelligence (AI)** and **Machine Learning (ML)** in modern agriculture. The system provides an intelligent decision-support platform that assists farmers in selecting the most suitable crops based on soil nutrients, environmental conditions, and real-time weather information. By integrating a trained machine learning model with the Flask web framework and the OpenWeatherMap API, the application delivers accurate crop recommendations, weather updates, fertilizer suggestions, and irrigation guidance through a simple and user-friendly web interface.

Throughout the project, each development phase was completed systematically, including data collection, preprocessing, machine learning model selection, backend development, frontend implementation, system integration, testing, and deployment. The application was designed with a modular architecture, making it reliable, scalable, and easy to maintain. Comprehensive testing confirmed that the system produces accurate predictions, responds efficiently to user requests, and provides an intuitive user experience across different devices

Demo link

<https://drive.google.com/file/d/1HVgCCgQEhUE8UINCuPy6GkkWbHqZvWjv/view?usp=sharing>